

DOCUMENTAZIONE DI PROGETTO

GRUPPO D4 - TASK R7: GESTIONE NOTIFICHE

Identificazione del Team e definizione degli obiettivi del progetto svolto, specificando:

ID del Team: D4

Nomi e E-mail dei componenti del Gruppo:

Pierluigi D'Angelo, pierl.dangelo@studenti.unina.it

Luca Esposito, luca.esposito52@studenti.unina.it

Salvatore Ferraro, salvatore.ferraro2@studenti.unina.it

Antonio Messina, antonio.messina6@studenti.unina.it

URL della Versione di Partenza sul repository (es. <https://github.com/Testing-Game-SAD-2023/A13>): <https://github.com/Testing-Game-SAD-2023/A13>

URL del Repository del progetto consegnato: <https://github.com/lukemuu/A13>

INDICE

1.	INTRODUZIONE	4
1.1	Descrizione sintetica del Task Assegnato	4
1.2	Obbiettivi del progetto	4
1.3	Analisi dell'impatto	4
2.	MESSAGE BROKER	6
2.1	Ruolo infrastrutturale del message broker	6
2.2	Benefici architetturali della comunicazione asincrona	7
2.3	Scelta del Broker: RabbitMQ	8
2.4	Topologia RabbitMQ	9
2.5	Pattern di comunicazione supportati.....	10
2.6	Contratti e DTO	12
2.7	Guida Operativa: integrare un microservizio con RabbitMQ	13
3.	SERVIZIO DI NOTIFICHE: TNOTIFICATION	16
3.1	Introduzione e scopo	16
3.2	Interfacce e interazione con gli altri servizi	16
3.3	Glossario dei termini utilizzati	16
3.4	Requisiti funzionali	17
3.5	Requisiti non funzionali	17
3.6	Casi d'uso	18
3.7	Scenari.....	18
3.8	Modello dei dati	22
3.9	Documentazione delle interfacce	23
3.9.1	Interfacce REST.....	23
3.9.2	Interfacce Asincrone (RabbitMQ).....	25
3.10	Package diagram.....	30
3.11	Sequence Diagram	34
4.	STRUTTURA FINALE DELL'ARCHITETTURA	39
4.1	Tabella delle modifiche	39

4.2 Component e Connectors diagram	40
4.3 Deployment Diagram	41
4.4 Descrizione dei test.....	42
4.4.1 Testing lato Backend (integrazione asincrona).....	42
4.4.2 Testing lato Frontend (API REST e SSE).....	42
4.5 Anti-pattern rilevati	46
5. Sviluppi futuri.....	48

1. INTRODUZIONE

1.1 Descrizione sintetica del Task Assegnato

Aggiungere una gestione generalizzata di notifiche di vario tipo da mostrare sulla Home Page del giocatore.

Il Task in questione richiederà lo studio e la progettazione di una generica soluzione per inviare notifiche fra parti diverse dell'architettura.

Realizzare un primo prototipo, da estendere poi successivamente.

1.2 Obiettivi del progetto

Il progetto si inserisce nel contesto di un'architettura software a microservizi in cui la comunicazione tra i vari componenti applicativi era già realizzata tramite API sincrone. Sebbene tale approccio garantisse semplicità di integrazione e immediatezza nelle interazioni dirette, esso risultava meno adeguato per la gestione di operazioni non critiche dal punto di vista temporale, introducendo un accoppiamento più forte tra i servizi e un potenziale impatto negativo sulle prestazioni complessive del sistema.

Il primo obiettivo del progetto è stato quindi quello di affiancare al modello di comunicazione sincrona esistente un meccanismo di comunicazione asincrona, capace di gestire quelle operazioni che non richiedono una risposta immediata. In tal senso è stato introdotto un message broker (RabbitMQ) che consente di demandare al sistema di messaggistica la gestione di eventi e notifiche, riducendo il carico sulle API sincrone e migliorando la scalabilità.

Il secondo obiettivo è stato la progettazione di un microservizio dedicato alla gestione delle notifiche utente. Tale componente utilizza sia meccanismi di comunicazione asincrona, basati su broker per la ricezione degli eventi provenienti dagli altri microservizi, sia comunicazioni sincrone tramite API, quando è richiesta un'interazione immediata con il microservizio di profilo utente (T5).

1.3 Analisi dell'impatto

L'introduzione del message broker comporta l'aggiunta di una nuova infrastruttura di comunicazione all'interno dell'architettura, senza richiedere modifiche strutturali ai microservizi già presenti. Il broker si inserisce come componente esterno, utilizzato

esclusivamente per specifici flussi comunicativi, senza alterare il funzionamento o le responsabilità dei servizi esistenti.

Tuttavia, i servizi che interagiranno con esso dovranno rispettare un modello di comunicazione condiviso, definito in funzione della tipologia di dati scambiati e della natura degli eventi, determinando strutture di messaggio differenti a seconda del contesto applicativo.

L'introduzione del microservizio di gestione delle notifiche incide sull'architettura esclusivamente in termini di ampliamento dei componenti presenti. Il servizio è indipendente dagli altri microservizi e non introduce dipendenze dirette, esponendo nuove funzionalità senza modificare il comportamento dei servizi esistenti.

2. MESSAGE BROKER

Nel modello sincrono tradizionale, un microservizio chiamante invoca direttamente un altro servizio e rimane in attesa della risposta, mantenendo occupato il thread di esecuzione fino al completamento dell'operazione. Questo comportamento introduce un accoppiamento temporale tra i servizi coinvolti, rendendo il sistema più sensibile a latenze, rallentamenti o indisponibilità parziali dei componenti chiamati.

L'adozione di un message broker consente di separare temporalmente il momento di produzione di un evento dal momento della sua elaborazione. Il servizio produttore pubblica un messaggio sul broker e può proseguire immediatamente la propria esecuzione, mentre uno o più servizi consumatori elaborano il messaggio in modo asincrono quando risultano disponibili. Questo modello abilita interazioni di tipo *fire-and-forget*, particolarmente adatte a casi d'uso che non necessitano dei risultati contenuti nella risposta al fine di proseguire l'elaborazione.

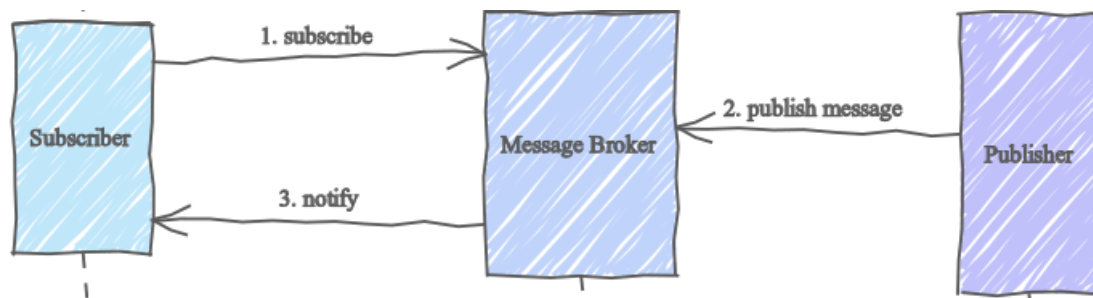


Figura 1: esempio di comunicazione tramite Broker.

2.1 Ruolo infrastrutturale del message broker

Il message broker introdotto nell'architettura svolge un ruolo esclusivamente infrastrutturale, dal momento che non incapsula logica di business né espone funzionalità riconducibili al dominio del sistema, ma si limita a fornire un supporto alla comunicazione tra i servizi.

Esso rappresenta uno strato di comunicazione condiviso e la sua responsabilità è circoscritta alla gestione dei messaggi, includendo l'instradamento, la persistenza e la consegna affidabile, senza alcuna conoscenza del significato semantico degli eventi.

All'interno dell'architettura, i microservizi interagiscono con il broker esclusivamente in qualità di produttori e consumatori di messaggi. Ogni servizio pubblica eventi sul broker e si sottoscrive ai flussi di interesse, evitando dipendenze dirette con gli altri servizi coinvolti nella comunicazione asincrona.

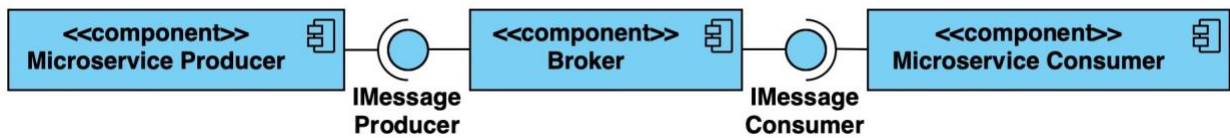


Figura 2: Vista Component & Connectors del Broker ed interfacce esposte ai Microservizi.

Dal punto di vista architetturale, l'utilizzo del broker consente di centralizzare aspetti puramente tecnici della comunicazione asincrona, quali:

- il routing dei messaggi verso le code o i topic appropriati;
- l'applicazione di meccanismi di retry e di gestione degli errori;
- l'osservabilità dei flussi di messaggistica.

Questa centralizzazione permette di applicare configurazioni e policy comuni senza introdurre logica aggiuntiva all'interno dei microservizi, mantenendo una netta separazione tra responsabilità infrastrutturali e responsabilità applicative.

2.2 Benefici architetturali della comunicazione asincrona

L'adozione di un message broker come intermediario tra i microservizi introduce una serie di benefici architetturali rilevanti, in particolare:

- **Disaccoppiamento temporale e logico**

In assenza di un broker, il servizio produttore dovrebbe conoscere l'endpoint del consumatore e attendere la sua risposta, rendendo l'interazione sensibile a problemi di latenza o indisponibilità.

Con l'introduzione del broker, il produttore non interagisce più direttamente con il consumatore, ma pubblica un messaggio su un canale intermedio. Il consumatore riceve il messaggio in modo asincrono, senza che il produttore debba conoscere la sua identità o il suo stato di disponibilità, favorendo una maggiore indipendenza tra i servizi.

- **Scalabilità orizzontale indipendente**

Il modello basato su broker consente ai servizi produttori e consumatori di scalare in modo indipendente. Nel caso di più istanze di uno stesso servizio consumatore, il broker è in grado di distribuire automaticamente i messaggi tra le istanze disponibili, implementando un meccanismo di **load balancing** nativo.

- **Affidabilità e resilienza**

I messaggi possono essere resi persistenti, garantendo la loro sopravvivenza anche in caso di riavvio del broker o dei servizi coinvolti. In presenza di consumatori

temporaneamente non disponibili, il broker conserva i messaggi e ne ritenta la consegna secondo politiche configurabili. Inoltre, i messaggi che non possono essere elaborati correttamente dopo un certo numero di tentativi sono instradati verso code dedicate (Dead Letter Queue), consentendo una successiva analisi degli errori.

Questo insieme di meccanismi contribuisce a rendere la comunicazione tra microservizi più robusta e tollerante ai guasti, riducendo l'impatto di errori locali sull'intero sistema.

2.3 Scelta del Broker: RabbitMQ

La scelta di RabbitMQ come message broker è stata determinata da un'analisi comparativa delle principali tecnologie disponibili, RabbitMQ e Kafka, in relazione alle specificità del progetto:

- **RabbitMQ** è indicato quando si devono gestire **eventi singoli, notifiche o task che richiedono consegna immediata** a uno o più consumatori, con possibilità di garantire retry e gestione degli errori.
- **Kafka** è più adatto quando si devono gestire **flussi continui di dati**, come log o eventi storici, con necessità di **ri-elaborazione o replay** dei messaggi in ordine cronologico.

In particolare, RabbitMQ si è rivelato il compromesso ideale grazie a:

- **Routing intelligente dei messaggi** utile a ridurre la necessità di logica di instradamento all'interno dei microservizi;
- **Prestazioni ottimizzate per singoli messaggi** con bassa latenza, adatte a scenari di notifiche, comandi e task individuali;
- **Integrazione nativa con Spring Boot** tramite Spring AMQP, un modulo del framework per la messagistica asincrona;
- **Gestione operativa semplificata**, mediante l'interfaccia web per il monitoraggio in tempo reale.

Per il nostro progetto, che prevede principalmente la **pubblicazione di notifiche e comandi tra microservizi**, RabbitMQ offre il giusto equilibrio tra **efficienza, semplicità di sviluppo e operatività**, risultando più immediato da integrare e gestire.

2.4 Topologia RabbitMQ

La comunicazione asincrona tramite RabbitMQ si basa su tre concetti fondamentali: **Exchange**, **Queue** e **Binding**. Comprendere il ruolo di questi elementi è essenziale per interpretare correttamente i diversi pattern di comunicazione supportati dal broker.

- **Exchange**

Rappresenta il punto di ingresso dei messaggi all'interno del broker. I produttori **non inviano mai messaggi direttamente alle code**, ma pubblicano i messaggi su un exchange specificando una *routing key*, cioè una stringa che identifica una o più code.

L'exchange ha il compito di **instradare** i messaggi verso una o più code, sulla base di regole di routing configurate, senza conoscere né il contenuto semantico del messaggio né i consumatori finali.

Possono essere di diversi **tipi**, in base al tipo di instradamento specifico:

1. **Direct Exchange**: instrada i messaggi verso code il cui binding corrisponde esattamente alla routing key.
2. **Fanout Exchange**: inoltra ogni messaggio a tutte le code collegate, ignorando la routing key.
3. **Topic Exchange**: instrada i messaggi in base a pattern di routing key, consentendo un filtraggio flessibile.

- **Queue**

Costituisce la struttura in cui i messaggi vengono effettivamente memorizzati. **Ogni coda è associata a uno o più consumer** (*competing consumers*), che prelevano i messaggi in modo asincrono e li elaborano secondo la propria logica di business.

La coda garantisce isolamento tra produttore e consumatore, persistenza dei messaggi (se configurata come durable) e consegna affidabile anche in caso di temporanea indisponibilità dei consumer.

Ogni messaggio instradato dall'exchange viene copiato all'interno di tutte le code per le quali il binding risulta compatibile con la routing key del messaggio.

- **Binding**

Rappresenta il collegamento logico tra un exchange e una queue. È all'interno del binding che viene definita la **regola di instradamento**, basata sulla routing key.

Dal punto di vista architetturale, il binding può essere visto come un **filtro applicato al flusso di messaggi** in uscita dall'exchange verso una specifica coda.

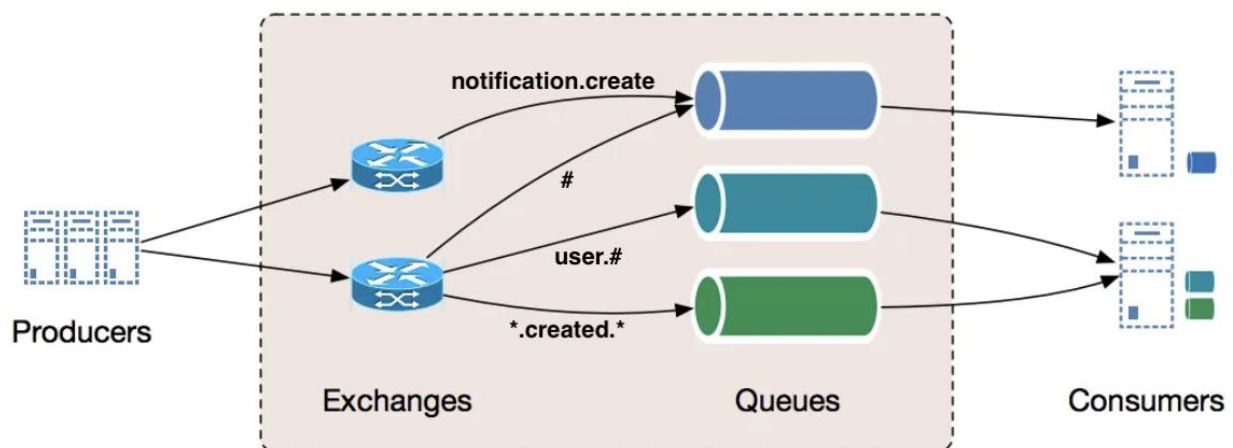


Figura 3: Esempio di Exchanges, Queues e Bindings in RabbitMQ.

2.5 Pattern di comunicazione supportati

A partire da questi elementi, RabbitMQ supporta diversi pattern di comunicazione asincrona:

- **Direct Exchange (1:1)**

Nel **Direct Exchange**, un messaggio viene instradato verso una coda **solo se la routing key del messaggio coincide esattamente** con quella definita nel binding con l'exchange. In questo caso ogni routing key identifica un flusso specifico e quindi il messaggio viene consegnato a una singola coda.

Questo pattern è adatto a comandi tra servizi, task specifici e richieste dirette (es. creazione di una notifica).

Esempio (**creazione notifica**):

Exchange: notifications.direct (tipo direct)

Routing key del messaggio: notification.create

Coda collegata (binding): notification.create

Il messaggio viene consegnato **solo alla coda destinata alla gestione delle notifiche**.

- **Fanout Exchange (1:N)**

Nel **Fanout Exchange**, l'exchange ignora completamente la routing key, quindi ogni messaggio pubblicato viene **replicato su tutte le code collegate** all'exchange.

Questo pattern risulta particolarmente adatto per eventi di sistema e notifiche broadcast.

Esempio:

Exchange: events.broadcast (tipo fanout)

Routing key del messaggio: qualsiasi (ignorata)

Code collegate (binding): *.created.*, user.#, #

Ogni messaggio pubblicato su events.broadcast viene **replicato su tutte le code collegate**, indipendentemente dalla routing key, quindi sia la coda *.created.* sia la coda user.# ricevono lo stesso messaggio.

- **Topic Exchange (1:Molti)**

Il Topic Exchange instrada i messaggi confrontando la **routing key del messaggio** con i **pattern definiti nei binding** delle code associate all'exchange.

I pattern di binding possono contenere le seguenti wildcard:

- * → una singola parola
- # → zero o più parole

Un messaggio viene consegnato a tutte le code il cui pattern di binding combacia con la routing key. In questo modo, ogni binding agisce come un **filtro** sugli eventi pubblicati, consentendo ai consumer di ricevere solo i sottoinsiemi di eventi di loro interesse.

Esempio:

Exchange: events.topic (tipo topic)

Routing key del messaggio: user.created.profile

Code collegate (binding): *.created.*, user.#, #

In questo caso tutte le code riceveranno il messaggio.

2.6 Contratti e DTO

Nella comunicazione asincrona tramite RabbitMQ, ogni messaggio pubblicato su un exchange deve rispettare un **contratto chiaro** tra produttore e consumatore.

Questo contratto è rappresentato da un **DTO (Data Transfer Object)**, che definisce la struttura dei dati scambiati tramite il broker, utile per garantire che **produttore e consumatore interpretino i dati nello stesso modo**, evitando errori di serializzazione o incomprensioni semantiche.

Esistono due approcci principali per la condivisione dei DTO:

1. Modulo o libreria condivisa

I DTO vengono pubblicati come package/libreria (ad esempio JAR) importabile dai microservizi.

Il vantaggio è quello di avere una tipizzazione forte e una compilazione sicura, ma si richiede la release coordinata della libreria quando il DTO cambia.

2. Specifica contrattuale indipendente

I DTO sono descritti tramite schemi formali (ad esempio JSON Schema, AsyncAPI) ed i team implementano i DTO localmente rispettando lo schema condiviso.

Si ha il vantaggio di avere completo decoupling tra team, ma sono richiesti controlli automatici per evitare discrepanze tra implementazioni.

Esempio:

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class NotificationDTO {
    private String userId;
    private String subject;
    private String body;
}
```

Figura 4: Esempio DTO.

Questo DTO rappresenta un messaggio di notifica inviato tramite il broker, quindi tutti i consumer che si sottoscrivono alla routing key corrispondente devono interpretare il messaggio secondo questa struttura.

2.7 Guida Operativa: integrare un microservizio con RabbitMQ

Questa guida mostra come un microservizio può **pubblicare e ricevere eventi/notifiche** tramite RabbitMQ, seguendo i contratti definiti dai DTO.

1. Connessione al broker e rete Docker

Per comunicare con RabbitMQ, ogni microservizio deve sapere **dove si trova il broker** e come connettersi. In un ambiente Docker, questo significa anche che il container del microservizio deve essere sulla **stessa rete** del broker per garantirne la visibilità.

```
services:
  vostro-servizio:
    networks:
      - global-network

networks:
  global-network:
    external: true
```

Figura 5: Esempio di rete Docker (docker-compose.yml).

```
spring:
  rabbitmq:
    host: tb-rabbitmq # Nome del container broker
    port: 5672        # Porta AMQP standard
    username: guest
    password: guest
```

Figura 6: Esempio di configurazione Spring (application.yml).

2. Dipendenze Maven

Per utilizzare RabbitMQ in Spring Boot, il progetto deve includere le seguenti dipendenze:

- **Spring Boot AMQP (spring-boot-starter-amqp):** gestisce tutto il collegamento al broker, serializzazione dei messaggi, listener, e template per invio/ricezione.

```
<!-- Spring Boot AMQP -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

Figura 7: Esempio pom.xml della dipendenza spring-amqp.

- **Lombok** (opzionale, ma consigliato): semplifica la definizione dei DTO evitando di scrivere manualmente getter, setter e costruttori.

```
<!-- Lombok (opzionale) -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
```

Figura 8: Esempio pom.xml della dipendenza lombok.

3. Configurazione del servizio

Ogni microservizio definisce opportunamente in una classe **code, exchange e binding** a seconda del pattern di comunicazione che vuole adottare.

```
@Bean
public Queue notificationQueue() {
    return new Queue("notifications.create.queue", true); // Nome della coda e persistenza
}

@Bean
public TopicExchange notificationExchange() {
    return new TopicExchange("notifications.exchange"); // Nome dell'exchange
}

@Bean
public Binding notificationBinding() {
    return BindingBuilder
        .bind(notificationQueue()) // Coda da collegare
        .to(notificationExchange()) // Exchange di destinazione
        .with("notifications.create"); // Routing key
}
```

Figura 9: Esempio Exchange dedicato al servizio notifiche

In questo esempio, il servizio notifiche riceverà solo i messaggi pertinenti alla creazione di nuove notifiche. Altri servizi possono definire exchange/coda propri o sottoscrivere allo stesso exchange se devono ricevere gli stessi eventi.

4. Creare un Producer

Il **Producer** consente al microservizio di pubblicare eventi o notifiche sul broker. Il messaggio viene inviato in modalità **fire-and-forget**, quindi il thread del servizio non resta bloccato in attesa di una risposta.

In particolare, si utilizza la classe `RabbitTemplate` per gestire automaticamente serializzazione e invio specificando il DTO con i dati dell'evento e la routing key che identifica la coda di destinazione.

```
@Service
public class EventProducerService {

    @Autowired
    private RabbitTemplate rabbitTemplate;

    /**
     * Pubblica un evento sul broker.
     *
     * @param exchange    Exchange su cui pubblicare il messaggio
     * @param routingKey  Routing key per instradare il messaggio
     * @param payload     DTO contenente i dati dell'evento
     */
    public void publishEvent(String exchange, String routingKey, Object payload) {
        rabbitTemplate.convertAndSend(exchange, routingKey, payload);
    }
}
```

Figura 10: Esempio di utilizzo di un Producer.

5. Creare un Consumer

Il Consumer permette di **ricevere eventi** pubblicati sul broker e di elaborarli nella logica di business.

In particolare, va definito un metodo annotato con `@RabbitListener` per ricevere direttamente il DTO, passandolo alla logica di business e sarà Spring ad occuparsi di estrarre automaticamente i messaggi dalla coda.

```
@Service
public class EventConsumerService {

    // Ascolta la coda specifica per le notifiche
    @RabbitListener(queues = "notifications.create.queue")
    public void handleEvent(NotificationDTO data) {
        processBusinessLogic(data);
    }

    private void processBusinessLogic(NotificationDTO data) {
        // Logica del servizio
    }
}
```

Figura 11: Esempio di utilizzo di un Consumer

3. SERVIZIO DI NOTIFICHE: TNOTIFICATION

3.1 Introduzione e scopo

Il servizio TNotification è stato progettato come **microservizio dedicato**, con l'obiettivo di centralizzare la creazione, la persistenza e la gestione delle notifiche utente.

Nel contesto dell'architettura, una **notifica utente** è un messaggio informativo associato ad un profilo utente, generato a seguito di un evento applicativo e reso disponibile al microservizio T5 per la presentazione verso l'interfaccia utente.

Questo microservizio viene introdotto al fine di separare la business logic legata alle notifiche da quella degli altri servizi, evitare dipendenze sincrone o rallentamenti all'interno dell'architettura dovuti alla generazione delle notifiche e, infine, garantire la persistenza dello storico e la disponibilità di queste ultime.

3.2 Interfacce e interazione con gli altri servizi

Il microservizio di notifiche si integra con il resto dell'architettura interagendo con altri microservizi per la creazione e gestione delle notifiche.

In particolare, mette a disposizione degli altri microservizi un'interfaccia per la **creazione** di notifiche generate in seguito a eventi applicativi del sistema, occupandosi della loro gestione e persistenza.

Il servizio si interfaccia inoltre con il microservizio utente per consentire la consultazione e la gestione delle notifiche associate a ciascun utente, includendo operazioni quali il **recupero** delle notifiche, l'**aggiornamento** del loro stato di lettura e la loro **eliminazione**.

I dettagli implementativi delle modalità di comunicazione e dei contratti di integrazione saranno definiti nelle successive fasi di progettazione.

3.3 Glossario dei termini utilizzati

Termine	Descrizione
Microservizio Utente	Microservizio dell'architettura che funge da punto di interfaccia tra l'utente e il sistema, occupandosi di ricevere le richieste provenienti dalla pagina web e di inoltrarle al servizio di gestione delle notifiche.
Microservizio	Altro microservizio dell'applicazione che avvia il processo di creazione della notifica.

3.4 Requisiti funzionali

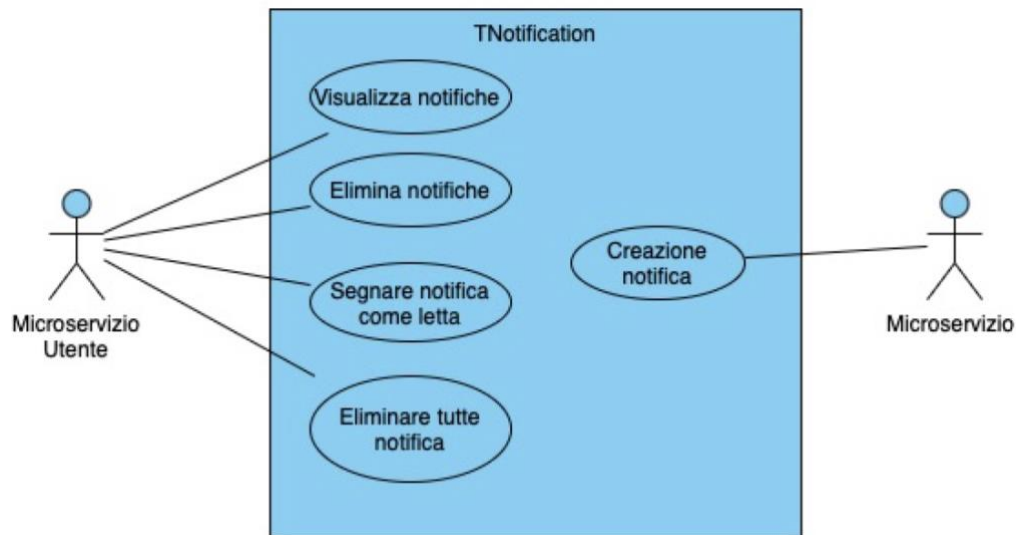
ID Requisito	Descrizione
RF-01	Il servizio deve poter permettere la creazione di messaggi di notifica utente su richiesta degli altri servizi dell'architettura.
RF-02	Il servizio deve poter permettere al Microservizio Utente la consultazione dello storico delle notifiche ricevute.
RF-03	Il servizio deve poter inviare aggiornamenti in tempo reale al Microservizio Utente.
RF-04	Il servizio deve poter permettere al Microservizio Utente di segnare una notifica come letta.
RF-05	Il servizio deve poter permettere al Microservizio Utente di filtrare le notifiche in base allo stato (letto/non letto).
RF-06	Il servizio deve poter permettere al Microservizio Utente di eliminare una notifica.
RF-07	Il servizio deve poter permettere al Microservizio Utente di cancellare tutte le notifiche associate ad un utente.

3.5 Requisiti non funzionali

RNF-01	La comunicazione con i Microservizi deve essere di tipo asincrono.
RNF-02	Il servizio deve poter garantire la persistenza delle notifiche.

3.6 Casi d'uso

A partire dai requisiti funzionali, è stato utilizzato il seguente Use Case diagram per indicare i casi d'uso e gli attori che interagiscono con il microservizio TNotification.



3.7 Scenari

Nelle tabelle che seguono sono illustrati gli scenari associati ai casi d'uso, al fine di evidenziare il contesto operativo e la sequenza di eventi principali.

- **Creazione Notifica**

Campo	Contenuto
Caso d'uso	Creazione Notifica
Attore primario	Microservizio
Descrizione	Un Microservizio richiede la creazione di una notifica per un utente.
PreCondizioni	- Il Microservizio conosce l'utente destinatario.
Sequenza di eventi principale	1. Il Microservizio richiede la creazione di una notifica. 2. Il servizio crea la notifica e la salva.

	2.1 In caso di errore, il servizio invia un messaggio di errore al Microservizio se il campo di risposta non è nullo. 3. Il servizio invia la notifica da far visualizzare all'utente. 4. Il servizio controlla se il campo di risposta è nullo. 4.1 Se il campo di risposta non è nullo, il servizio invia un messaggio di conferma dell'operazione al Microservizio.
PostCondizioni	- La notifica è stata creata e salvata correttamente.

- **Visualizza notifiche**

Campo	Contenuto
Caso d'uso	Visualizza notifiche
Attore primario	Microservizio Utente
Descrizione	Il Microservizio Utente richiede la lista delle notifiche relative ad un utente, eventualmente filtrate per stato o tipo.
PreCondizioni	- Il servizio è operativo.
Sequenza di eventi principale	1. Il Microservizio Utente richiede la lista delle notifiche. 2. Il servizio recupera la lista e applica eventuali filtri. 3. Il servizio restituisce la lista al Microservizio Utente.
PostCondizioni	- Il Microservizio Utente riceve la lista delle notifiche richieste.

- **Segna notifica come letta**

Campo	Contenuto
Caso d'uso	Segna notifica come letta
Attore primario	Microservizio Utente
Descrizione	Il Microservizio Utente richiede di marcare una specifica notifica come letta.
PreCondizioni	- Il servizio è operativo. - Esiste una notifica da marcare come letta.
Sequenza di eventi principale	1. Il Microservizio Utente richiede di marcare una notifica come letta. 2. Il servizio verifica l'esistenza della notifica. 2.1 Se la notifica non esiste genera un errore e lo inoltra al Microservizio Utente 3. Il servizio aggiorna lo stato della notifica. 4. Il servizio registra l'aggiornamento.
PostCondizioni	- La notifica risulta marcata come letta.

- **Elimina notifica**

Campo	Contenuto
Caso d'uso	Elimina notifica
Attore primario	Microservizio Utente
Descrizione	Il Microservizio Utente richiede l'eliminazione di una specifica notifica.

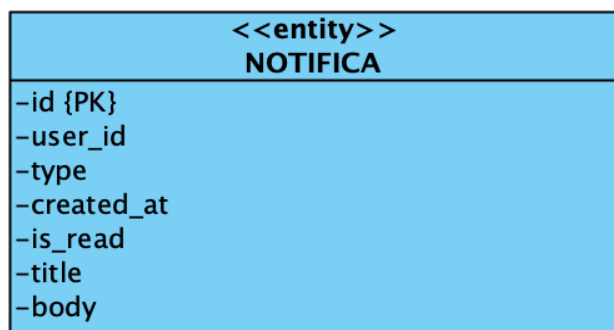
PreCondizioni	- Il servizio è operativo. - Esiste una notifica da eliminare.
Sequenza di eventi principale	1. Il Microservizio Utente richiede di eliminare una notifica. 2. Il servizio verifica l'esistenza della notifica. 2.2 Se la notifica non esiste il servizio genera un errore e lo inoltra al Microservizio Utente 3. Il servizio rimuove la notifica.
PostCondizioni	- La notifica è stata eliminata.

- **Elimina tutte le notifiche**

Campo	Contenuto
Caso d'uso	Elimina tutte le notifiche
Attore primario	Microservizio Utente
Descrizione	Il Microservizio Utente richiede l'eliminazione di tutte le notifiche relative ad un utente.
PreCondizioni	- L'utente è autenticato. - Esistono notifiche da eliminare.
Sequenza di eventi principale	1. Il Microservizio Utente richiede di eliminare tutte le notifiche. 2. Il sistema verifica la richiesta. 3. Il sistema rimuove tutte le notifiche dell'utente.
PostCondizioni	- Tutte le notifiche sono state eliminate.

3.8 Modello dei dati

Il microservizio gestisce unicamente **notifiche utente**, modellate come entità secondo il seguente schema:



Campo	Tipo	Vincoli / Proprietà	Descrizione
Id	BIGSERIAL	PK, NOT NULL	Identificativo univoco della notifica
user_id	BIGINT	NOT NULL	Identificativo dell'utente destinatario della notifica
Type	VARCHAR (100)	NOT NULL	Tipologia della notifica
created_at	TIMESTAMP	NOT NULL, non aggiornabile	Data e ora di creazione della notifica
is_read	BOOLEAN	NOT NULL, default false	Indica se la notifica è stata letta dall'utente o meno
title	VARCHAR (200)	NOT NULL	Titolo della notifica, utilizzato come riepilogo del contenuto
body	TEXT	NOT NULL	Contenuto testuale completo della notifica

Al fine di soddisfare il requisito RNF-02, il servizio mantiene un **database PostgreSQL** dedicato, utilizzato esclusivamente per la persistenza delle notifiche affinché sia rispettata l'autonomia dei singoli servizi all'interno delle buone norme di programmazione a microservizi.

Tale scelta è stata adottata in previsione di possibili evoluzioni future del modello dei dati, mediante l'impiego di un opportuno campo di tipo *'JSONB'*, che consente di introdurre agevolmente attributi aggiuntivi dipendenti dalla specifica tipologia di notifica, senza compromettere né alterare la struttura relazionale del database.

3.9 Documentazione delle interfacce

3.9.1 Interfacce REST

Le operazioni di lettura, modifica dello stato ed eliminazione delle risorse sono state implementate tramite interfacce sincrone conformi ai principi REST, in quanto caratterizzate dalla necessità di fornire al client un riscontro immediato sull'esito dell'operazione. Inoltre, è stato utilizzato lo standard Server-Sent Events (SSE) per la comunicazione real-time verso i client.

Documentazione (OpenAPI 3.0)

Per garantire la standardizzazione e l'interoperabilità con il Frontend e l'API Gateway, l'interfaccia è stata formalizzata utilizzando la specifica OpenAPI 3.0. L'adozione di un approccio *Code-First* (tramite la libreria *springdoc-openapi*) ha permesso di generare automaticamente la documentazione tecnica a partire dal codice sorgente, garantendo che le specifiche siano sempre sincronizzate con l'implementazione effettiva del Controller.

La documentazione interattiva è accessibile tramite Swagger UI, strumento che permette agli sviluppatori e ai tester di esplorare gli endpoint e simulare richieste direttamente dal browser.

Catalogo degli Endpoint

Il servizio espone le seguenti risorse, mappate sul path base `/notifications`.

Notifications Gestione delle notifiche utente e sottoscrizione SSE			^
PATCH	<code>/notifications/{userId}/{notificationId}/read</code>	Segna una notifica come letta	▼
GET	<code>/notifications/{userId}</code>	Recupera tutte le notifiche di un utente	▼
DELETE	<code>/notifications/{userId}</code>	Elimina tutte le notifiche di un utente	▼
GET	<code>/notifications/{userId}/{notificationId}</code>	Recupera una singola notifica	▼
DELETE	<code>/notifications/{userId}/{notificationId}</code>	Elimina una notifica	▼
GET	<code>/notifications/subscribe/{userId}</code>	Sottoscrizione SSE alle notifiche	▼

Figura 12: Catalogo degli end-points nella UI Swagger

Modello dei Dati (Schema)

La struttura dei dati scambiati tramite le API è definita formalmente nello schema `NotificationRestDTO`, visibile nella documentazione Swagger. Questo oggetto rappresenta il contratto unico utilizzato sia per le risposte REST sincrone che per gli eventi asincroni SSE, garantendo coerenza nel parsing lato frontend.

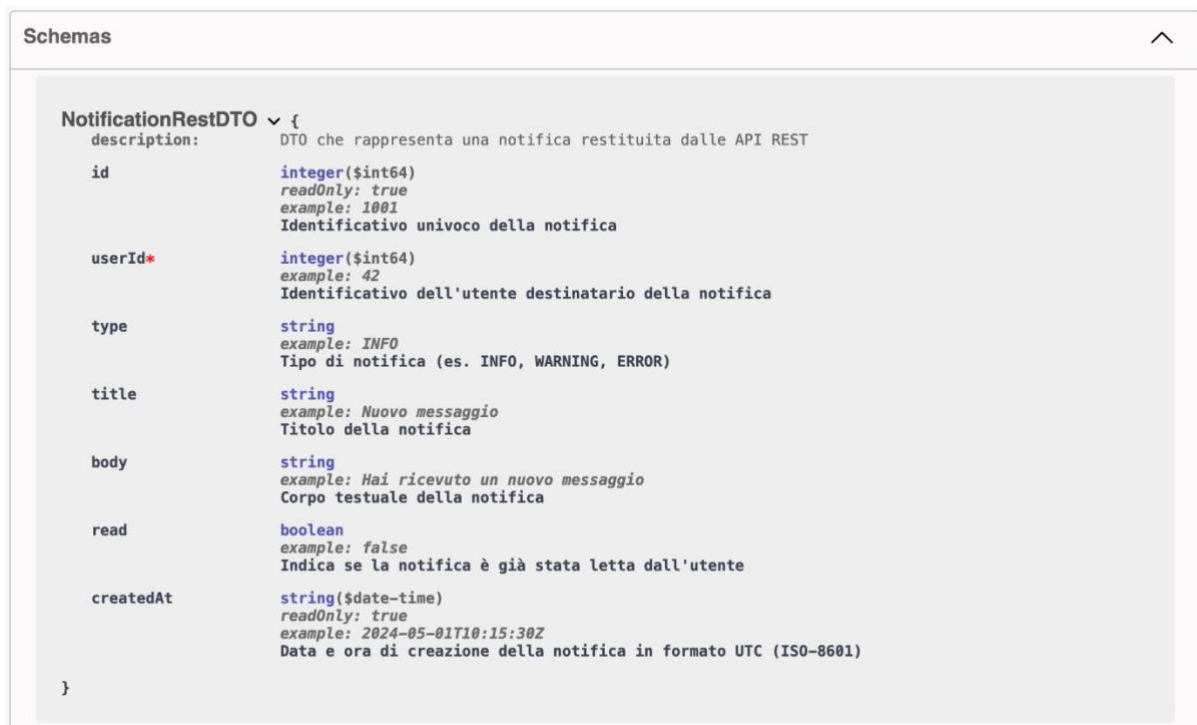


Figura 22: Catalogo degli end-points nella UI Swagger.

Gestione Real-Time (Server-Sent Events)

A differenza delle tradizionali chiamate REST sincrone, l'endpoint `/subscribe` implementa un pattern di comunicazione asincrono unidirezionale (Server-to-Client).

Il client stabilisce una connessione persistente con `Content-Type: text/event-stream`. Il server mantiene attivo il canale (`SseEmitter`) e invia i payload JSON non appena una nuova notifica viene consumata dal message broker, eliminando la necessità di *polling* continuo da parte del frontend.

3.9.2 Interfacce Asincrone (RabbitMQ)

L'operazione di creazione della notifica è stata realizzata mediante una logica asincrona, al fine di consentire agli altri servizi di inoltrare la richiesta senza dover sospendere l'esecuzione della propria logica di business.

Il microservizio opera come componente *Consumer* all'interno dell'architettura a eventi, ricevendo messaggi da altri servizi del sistema (Producer) tramite il message broker RabbitMQ.

Documentazione (AsyncAPI)

Per formalizzare il contratto di comunicazione asincrona ed evitare disallineamenti tra i team, è stata adottata la specifica AsyncAPI, che descrive in modo rigoroso:

- Canali: Le code e gli Exchange su cui viaggiano i messaggi.
- Messaggi: La struttura del payload (Schema JSON).
- Protocollo: Le specifiche di connessione AMQP.

Tutto ciò permette al team di sviluppo di visualizzare la topologia completa dei messaggi e validare i payload prima dell'implementazione effettiva.

Schema del Messaggio (DTO)

Il payload dei messaggi in ingresso è definito dall'oggetto NotificationDTO. La struttura include sia i dati funzionali per l'utente (titolo, corpo) sia i metadati di routing (replyTo).

NotificationDTO UNSUPPORTED-PROTOCOL RECEIVE

Channel notifications.create.queue

Operation description Elabora notifiche asincrone ricevute sulla coda notifications.create.queue

Message description Oggetto di trasferimento dati per la creazione di una notifica

Content-Type application/json

Servers rabbitmq

Operation Binding

Message Binding

Headers

HeadersNotDocumented

Payload

NotificationDTO

- ▶ **body** string
Contenuto testuale completo della notifica
Example: Grazie per esserti registrato alla nostra piatta...
- ▶ **replyTo** string
Nome della coda RabbitMQ su cui inviare la risposta (opzionale)
Example: reply.queue.service.t4
- ▶ **title** string
Titolo breve della notifica
Example: Benvenuto!
- ▶ **type** string
Categoria della notifica (es. INFO, WARNING, ERROR)
Example: INFO
- ▶ **userId*** integer (int64)
Identificativo univoco dell'utente destinatario
Example: 101

```
{  "body": "Grazie per esserti registrato alla nostra  piattaforma.",  "replyTo": "reply.queue.service.t4",  "title": "Benvenuto!",  "type": "INFO",  "userId": 101}
```

Copy Reset Publish

Figura 33: Schema del DTO sull'UI Springwolf.

Libreria Client Condivisa (Pattern Software Development Kit)

Al fine di disaccoppiare i microservizi chiamanti dalla complessità del protocollo AMQP, è stata realizzata una Libreria Client Condivisa, distribuita come dipendenza Maven.

L'adozione di questo pattern architetturale (Client SDK) offre vantaggi significativi in termini di manutenibilità e governance:

1. Incapsulamento: I dettagli infrastrutturali (nomi degli Exchange, Routing Key, logiche di serializzazione JSON) sono nascosti all'interno della libreria.
2. Evolutività: Eventuali modifiche alla topologia di RabbitMQ richiedono solo l'aggiornamento della libreria centralizzata, senza necessità di refactoring del codice di business nei servizi consumatori.
3. Standardizzazione: Tutti i microservizi utilizzano i medesimi DTO e le stesse policy di gestione degli errori.

Questa libreria espone l'interfaccia `NotificationProducer`, che agisce come *Facade*. I servizi produttori devono semplicemente iniettare questa interfaccia per inviare notifiche, senza conoscere i dettagli infrastrutturali.

```
import com.a13.notification.client.service.NotificationProducer;
import com.a13.notification.client.dto.NotificationDTO;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class AchievementService {
    @Autowired
    private NotificationProducer producer;

    public void notifyAchievementUnlocked(Long userId, String achievementName) {
        NotificationDTO dto = new NotificationDTO(
            userId,
            "SUCCESS",
            "Achievement Sbloccato",
            String.format("Complimenti! Hai sbloccato l'achievement '%s'. Continua così!", achievementName),
            null
        );
        producer.sendCreateNotification(dto); // Invia su exchange topic
    }
}
```

Figura 14: Esempio di utilizzo della libreria condivisa

Interazione asincrona tra i microservizi e il servizio di notifiche

L'architettura di messaggistica è progettata per garantire scalabilità e flessibilità, in particolare prevede:

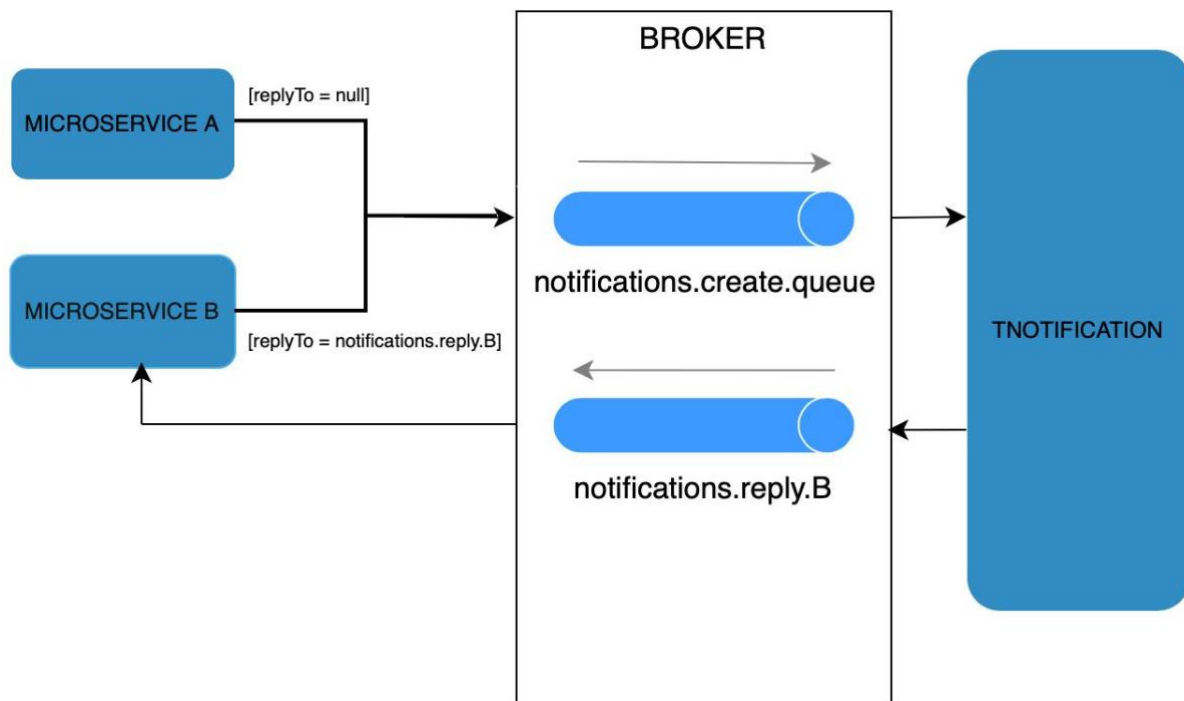


Figura 15: Schema di comunicazione tra microservizi e servizio di notifiche.

- **Coda di Ingresso condivisa:**

Tutti i messaggi di notifica convergono su un'unica coda persistente (`notifications.create.queue`), collegata all'exchange topic `notifications.exchange` con routing key dedicata (`notifications.create`). Questo design permette la scalabilità orizzontale: multiple istanze del microservizio TNotification possono consumare messaggi in parallelo dalla stessa coda, dividendosi il carico di lavoro.

- **Coda di risposta (opzionale):**

Il sistema supporta un meccanismo di *callback* dinamico tramite il campo `replyTo` del DTO, tramite il quale distinguere i casi in cui è richiesta una risposta o meno dal produttore della richiesta di creazione della notifica.

Si distinguono pertanto due possibili modalità:

1. **Fire-and-Forget:** Se il campo `replyTo` è nullo (configurazione di default), il servizio elabora la notifica senza inviare conferme, ottimizzando il throughput.
2. **RPC Asincrono:** Se il servizio chiamante abilita la modalità di risposta, il servizio Notification legge il campo `replyTo` che gli indicherà la specifica coda dove mandare la risposta, permettendo al chiamante di ricevere una conferma di avvenuta elaborazione.

Dal punto di vista del servizio produttore, la libreria valorizza automaticamente il campo `replyTo` leggendo il nome della coda di callback dalle proprietà di configurazione (`application.properties`).

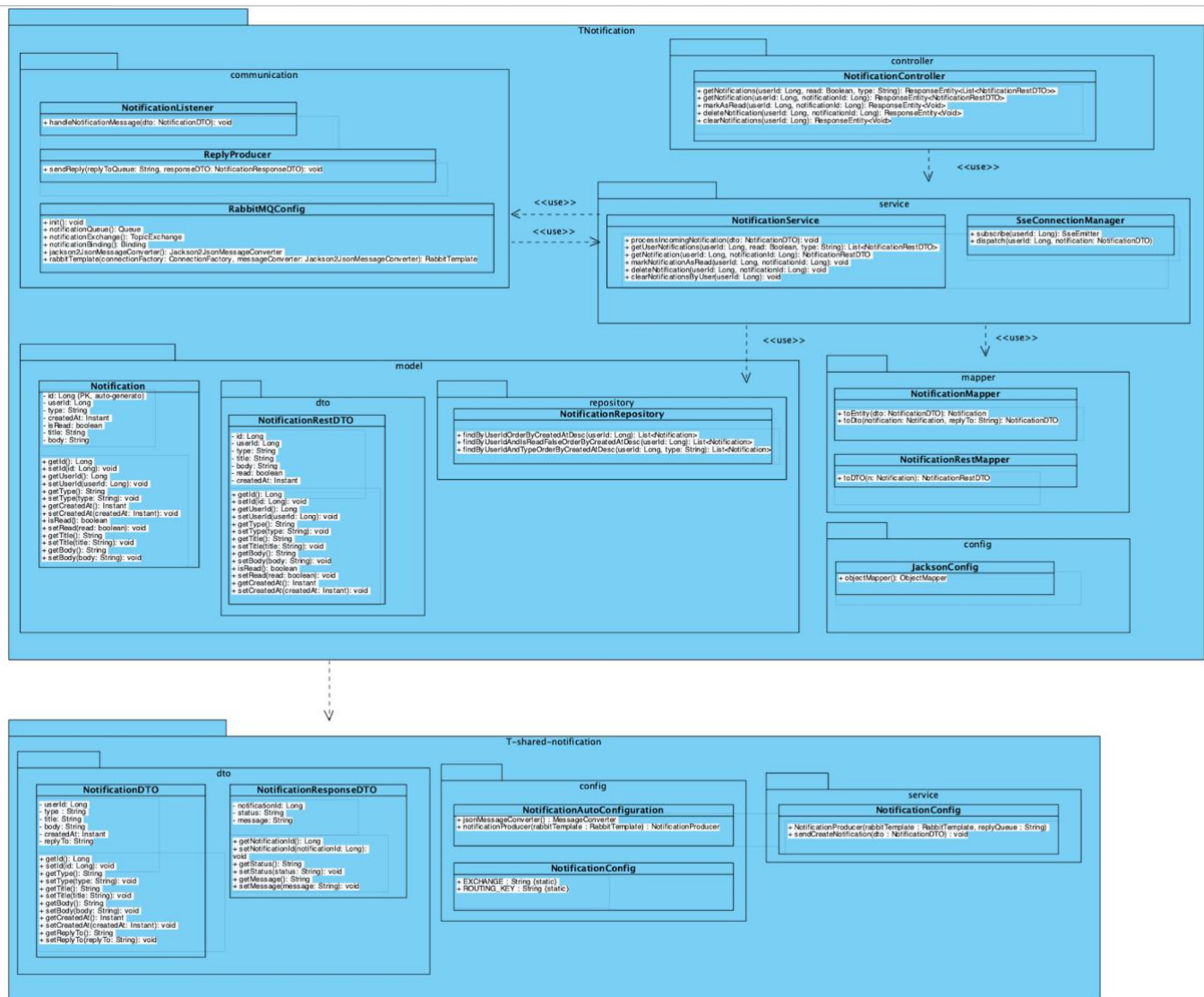
```
notification.reply.queue=t10.reply.queue
```

Figura 15: Esempio configurazione (in `application.properties` del chiamante).

In assenza di questa proprietà verrà utilizzata di default la modalità *fire-and-forget*.

3.10 Package diagram

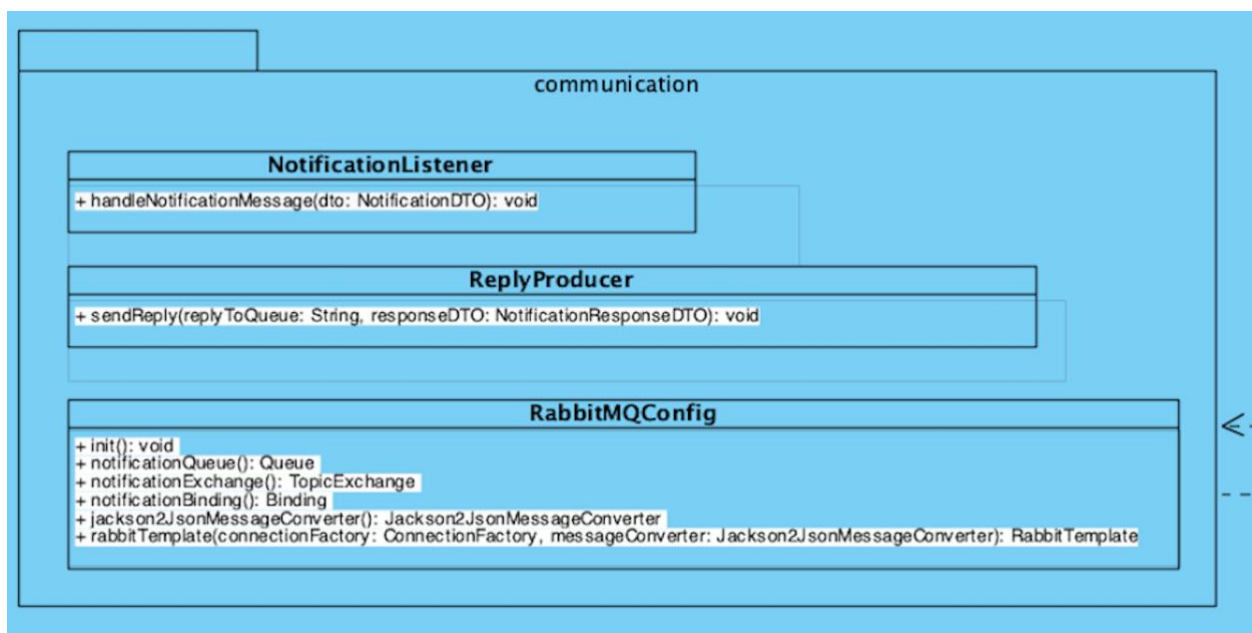
Il seguente package diagram mostra una vista statica della soluzione di dettaglio nella progettazione del servizio *TNotification*. In particolare, è stato adottato il pattern standard di Spring, articolando il codice in package quali *model*, *service* e *controller*. Risulta invece assente il package *view*, in quanto il servizio è concepito come componente operativo esclusivamente backend.



Nel diagramma riportato si individuano i seguenti packages:

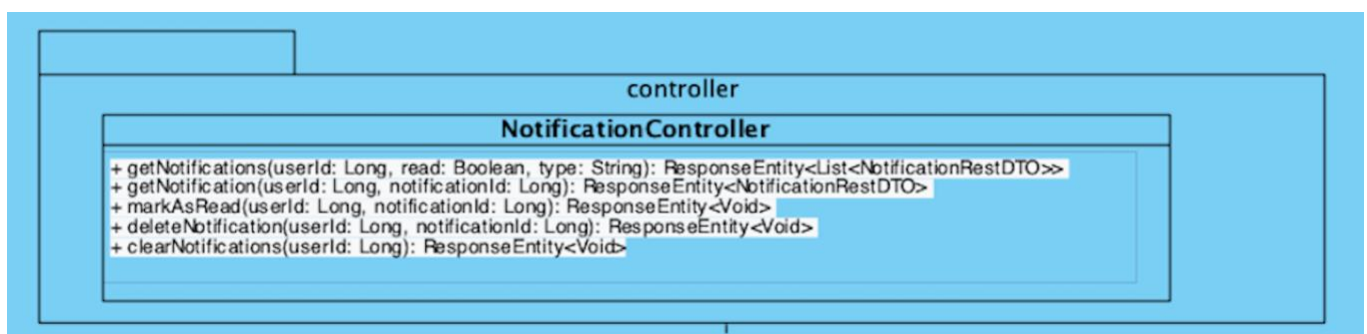
Communication:

Gestisce l'integrazione con RabbitMQ tramite RabbitMQConfig (infrastruttura e JSON), NotificationListener (ricezione notifiche) e ReplyProducer (invio feedback), assicurando il flusso completo dei messaggi tra il broker e la logica applicativa.



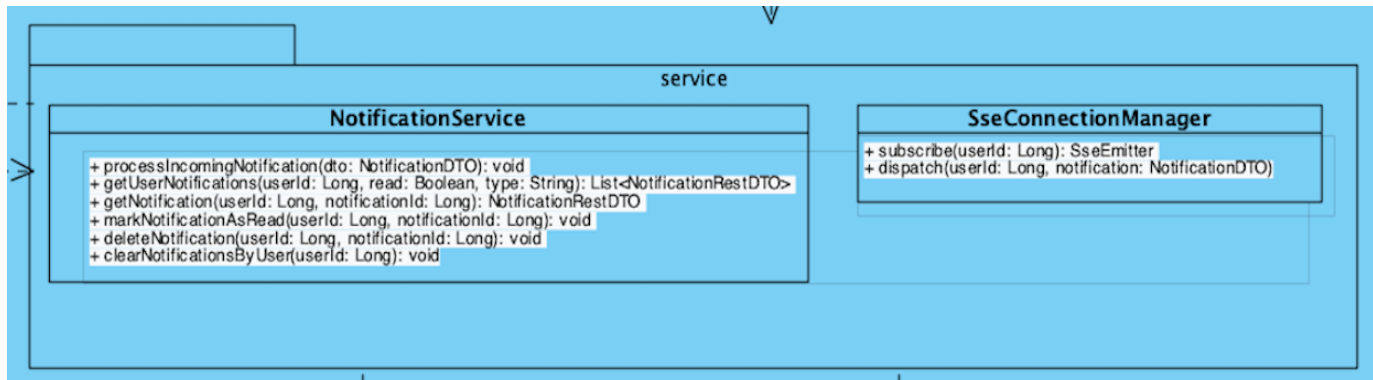
Controller:

Espone le API REST e gli endpoint SSE (Server-Sent Events) tramite la classe NotificationController per permettere agli utenti di gestire le notifiche e riceverle in tempo reale.



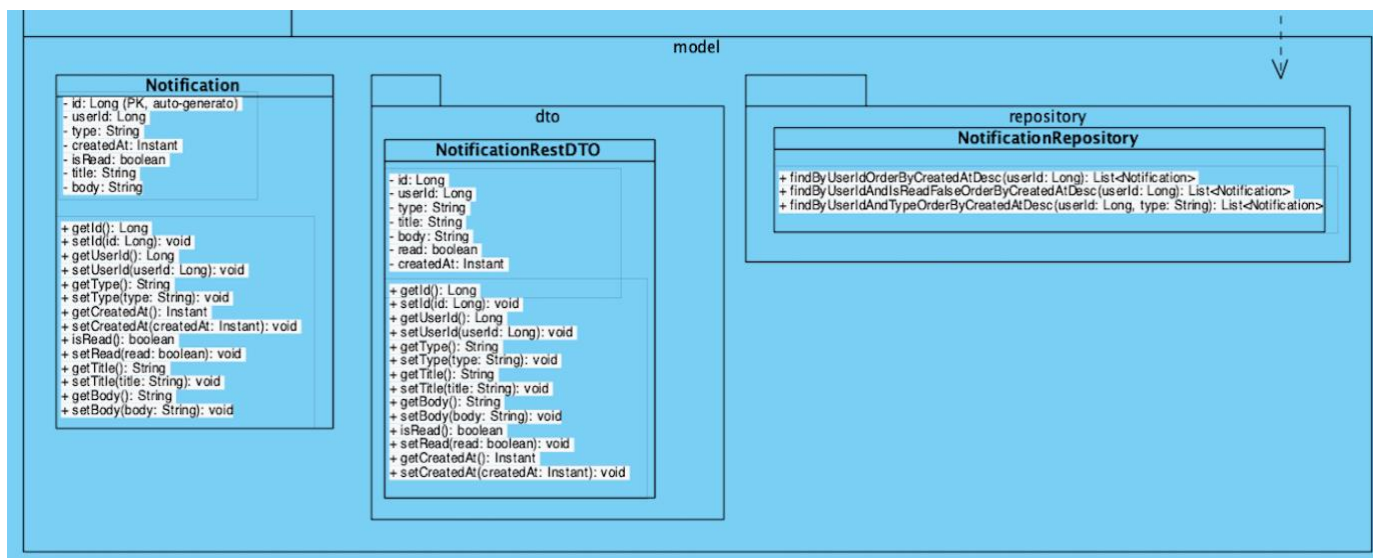
Service:

Implementa la logica di business coordinando il salvataggio delle notifiche su database, la gestione delle sottoscrizioni e degli invii in tempo reale tramite SSE.



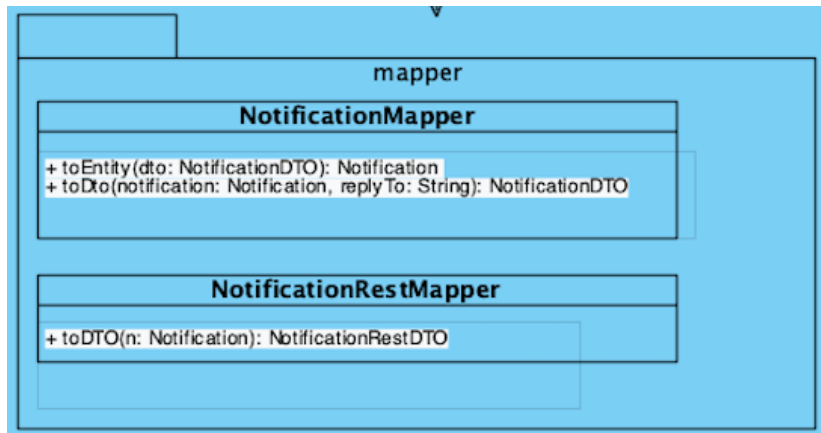
Model:

Definisce lo strato di persistenza e i formati di scambio dati, contenendo l'entità **Notification**, il **NotificationRepository** per l'interazione con il database (CRUD) e il **NotificationRestDTO** per la rappresentazione delle risorse nelle API REST.



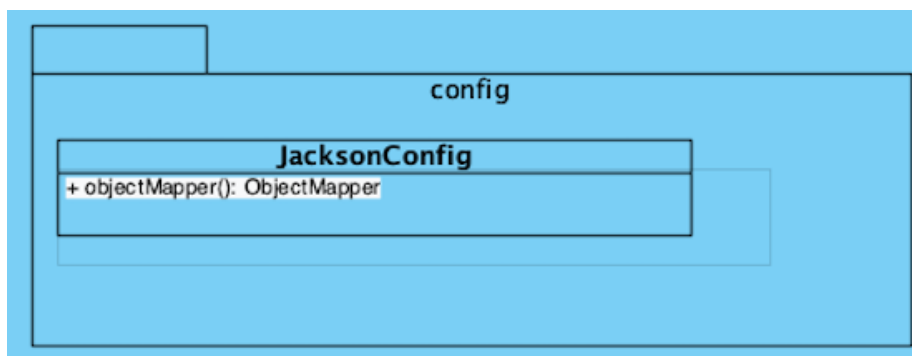
Mapper:

Contiene le classi responsabili della conversione dei dati tra l'entità Notifica e i diversi formati DTO (NotificationDTO e NotificationRestDTO)



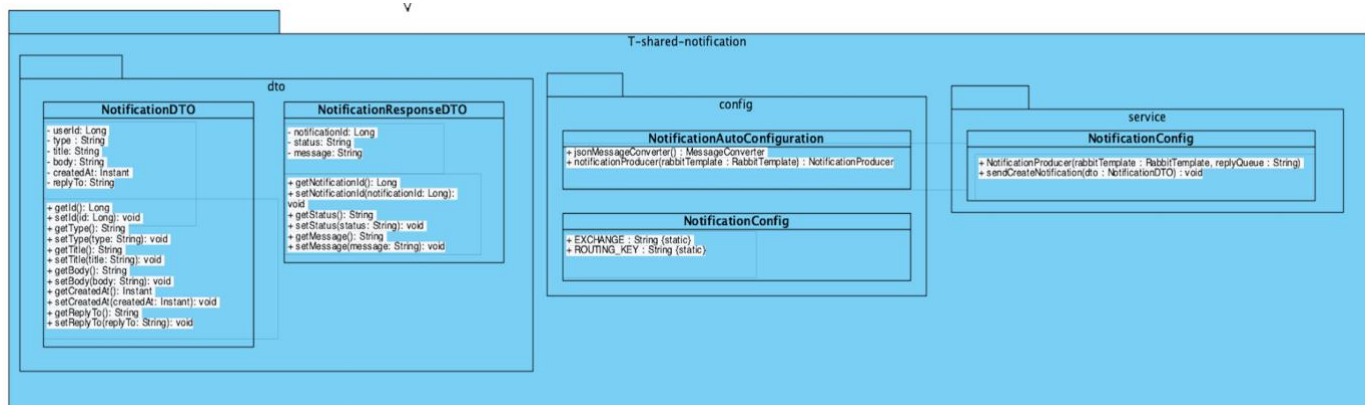
Config:

Configura Jackson per convertire gli oggetti Java in JSON e viceversa, garantendo tramite il `JavaTimeModule` la corretta gestione dei campi temporali come `Instant`.



T-shared-notification:

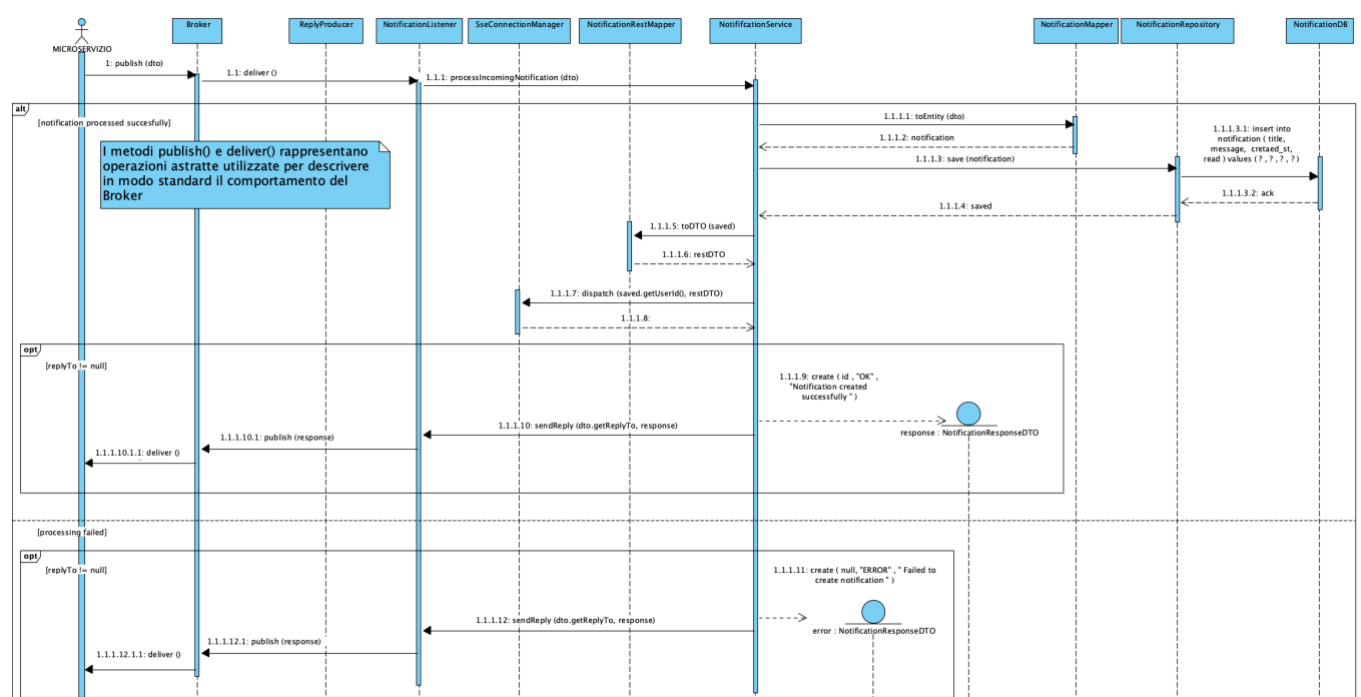
Libreria che incapsula la complessità di RabbitMQ, esponendo un'interfaccia standard per disaccoppiare i microservizi dall'infrastruttura e garantire uniformità nello scambio dei dati.



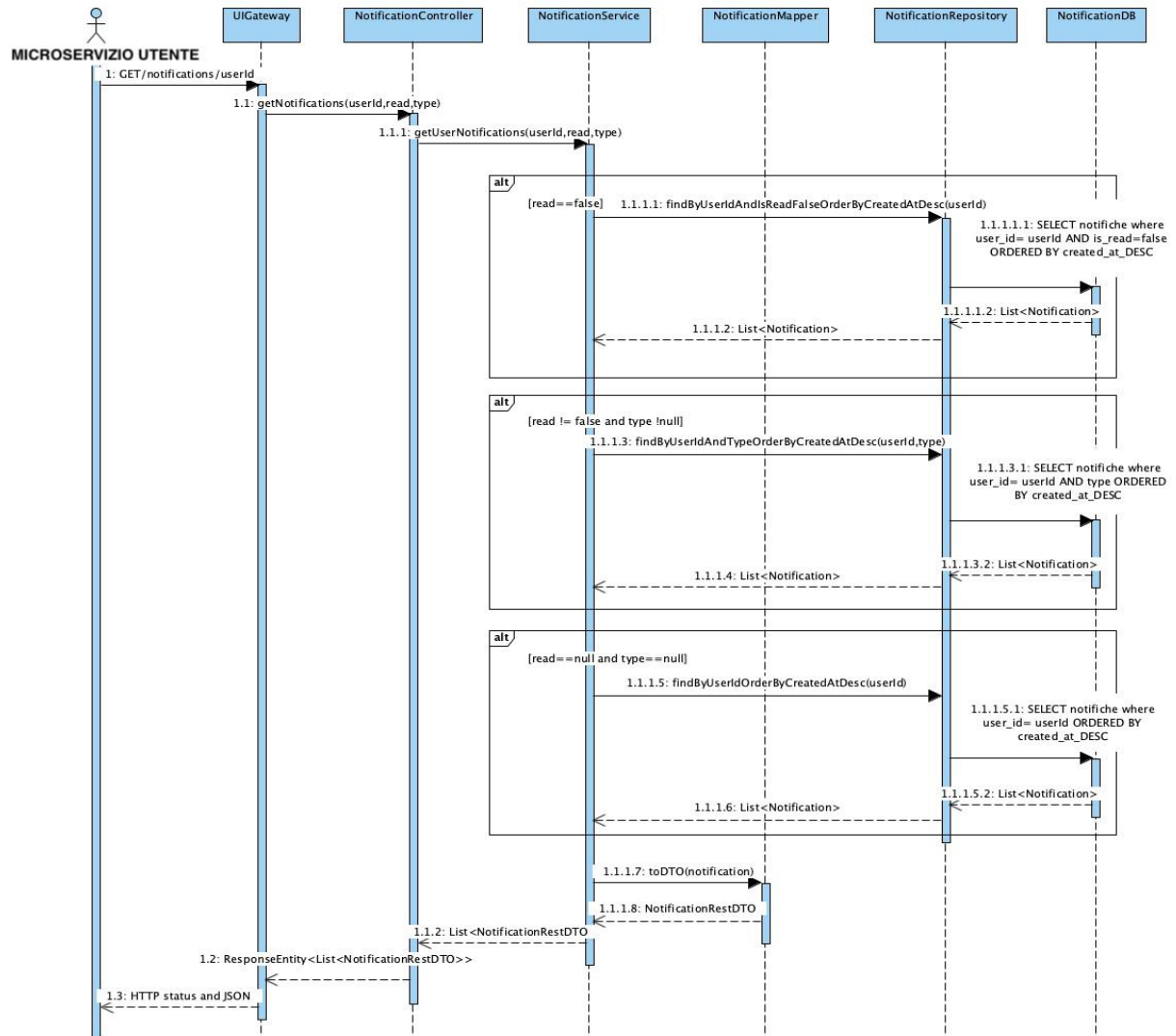
3.11 Sequence Diagram

I sequence diagram riportati di seguito offrono una vista dinamica sulla progettazione di dettaglio del servizio `TNotification`, evidenziando le possibili operazioni supportate e le interazioni tra le diverse classi appartenenti ai package del servizio.

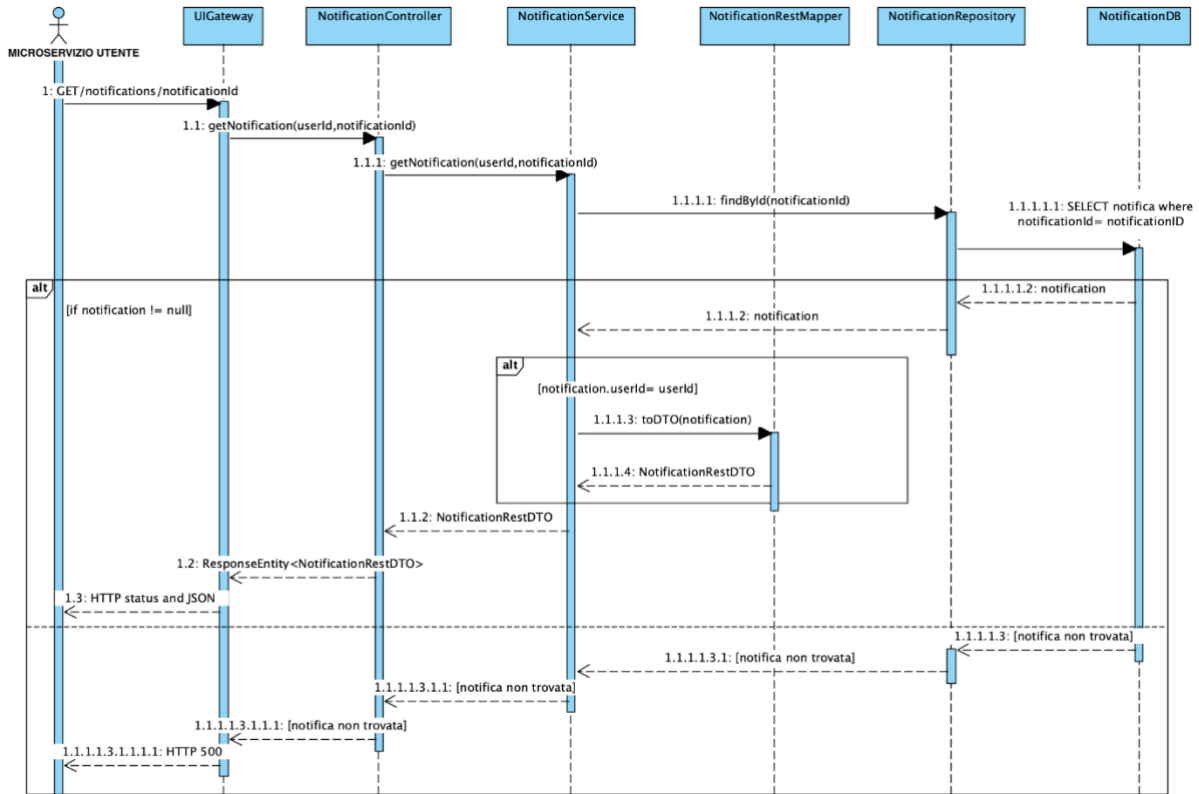
• Create Notification



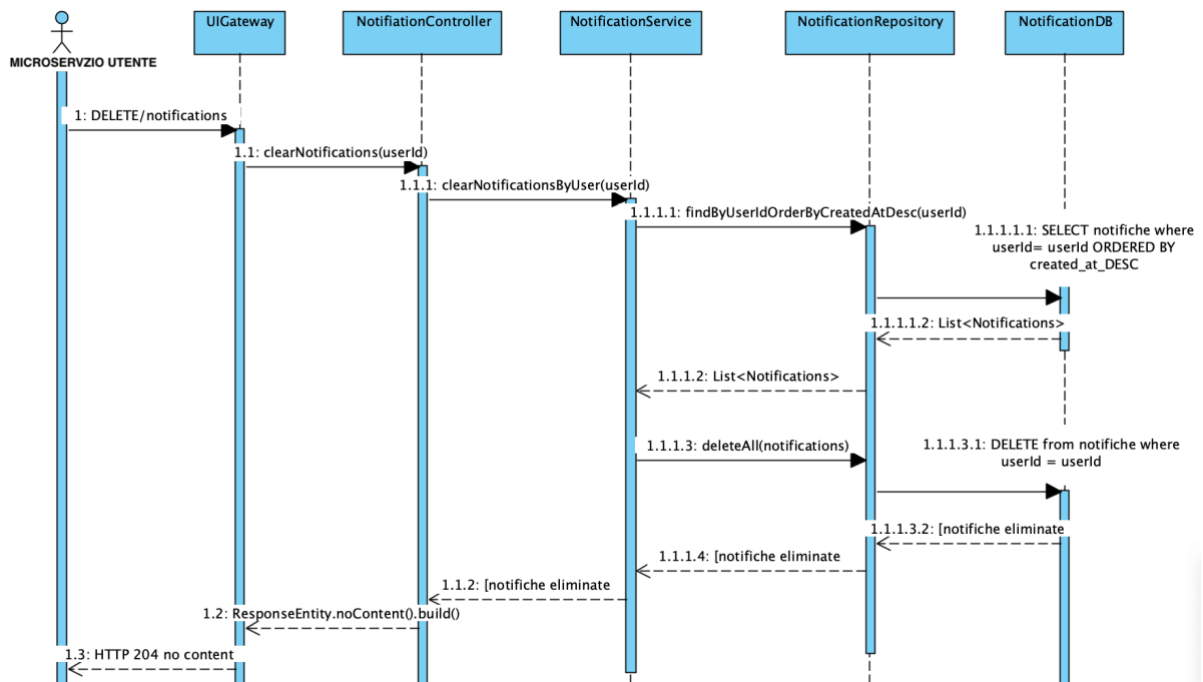
- Get All Notification



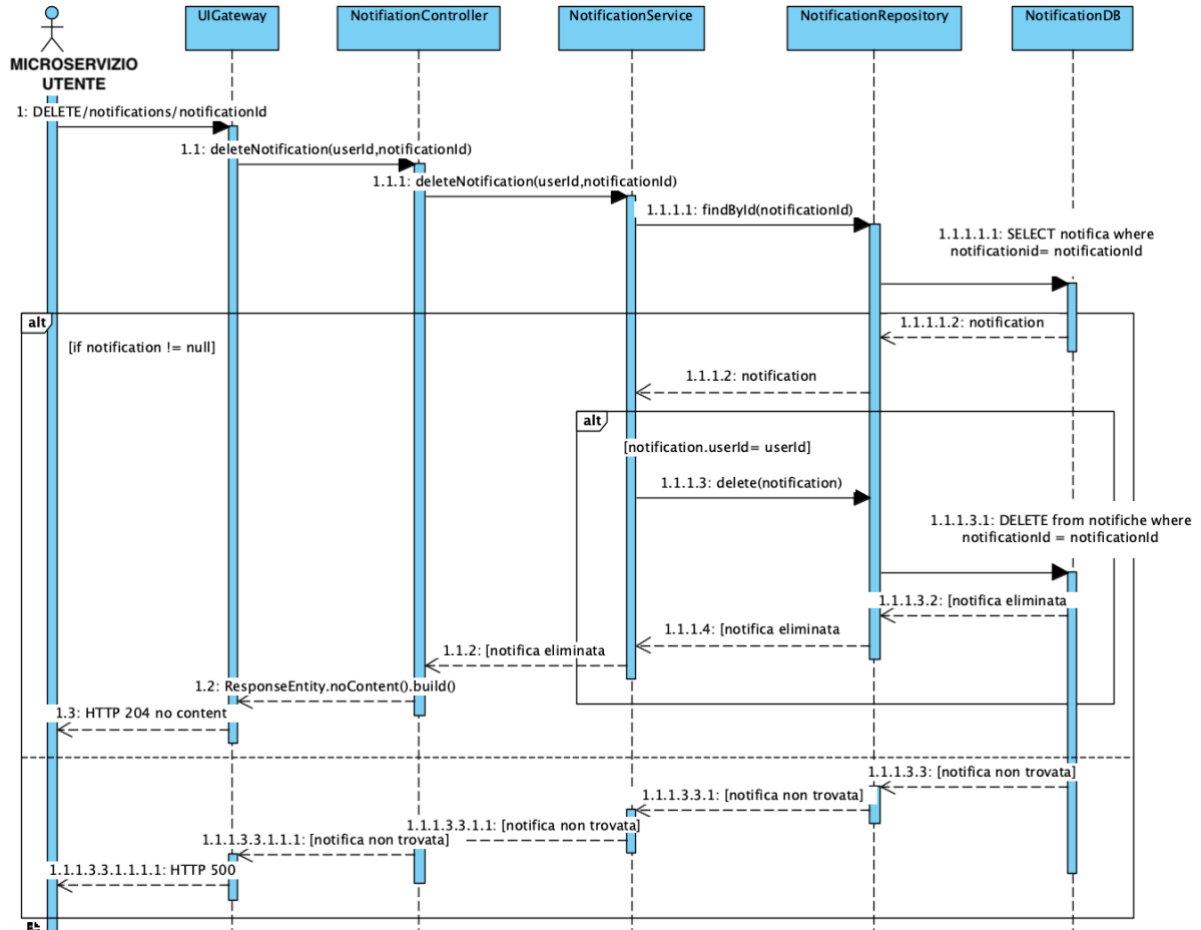
- Get Notification



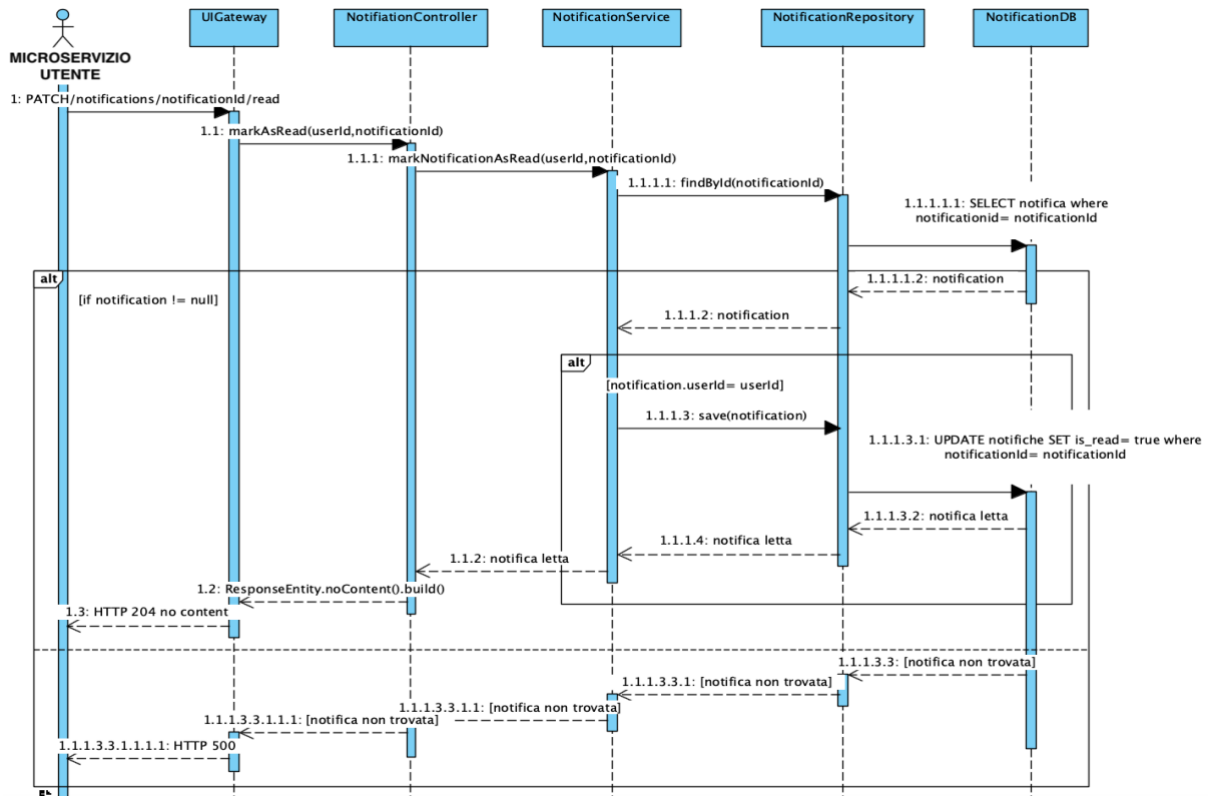
- Delete All Notification



- Delete Notification



- Mark Notification as Read



4. STRUTTURA FINALE DELL'ARCHITETTURA

4.1 Tabella delle modifiche

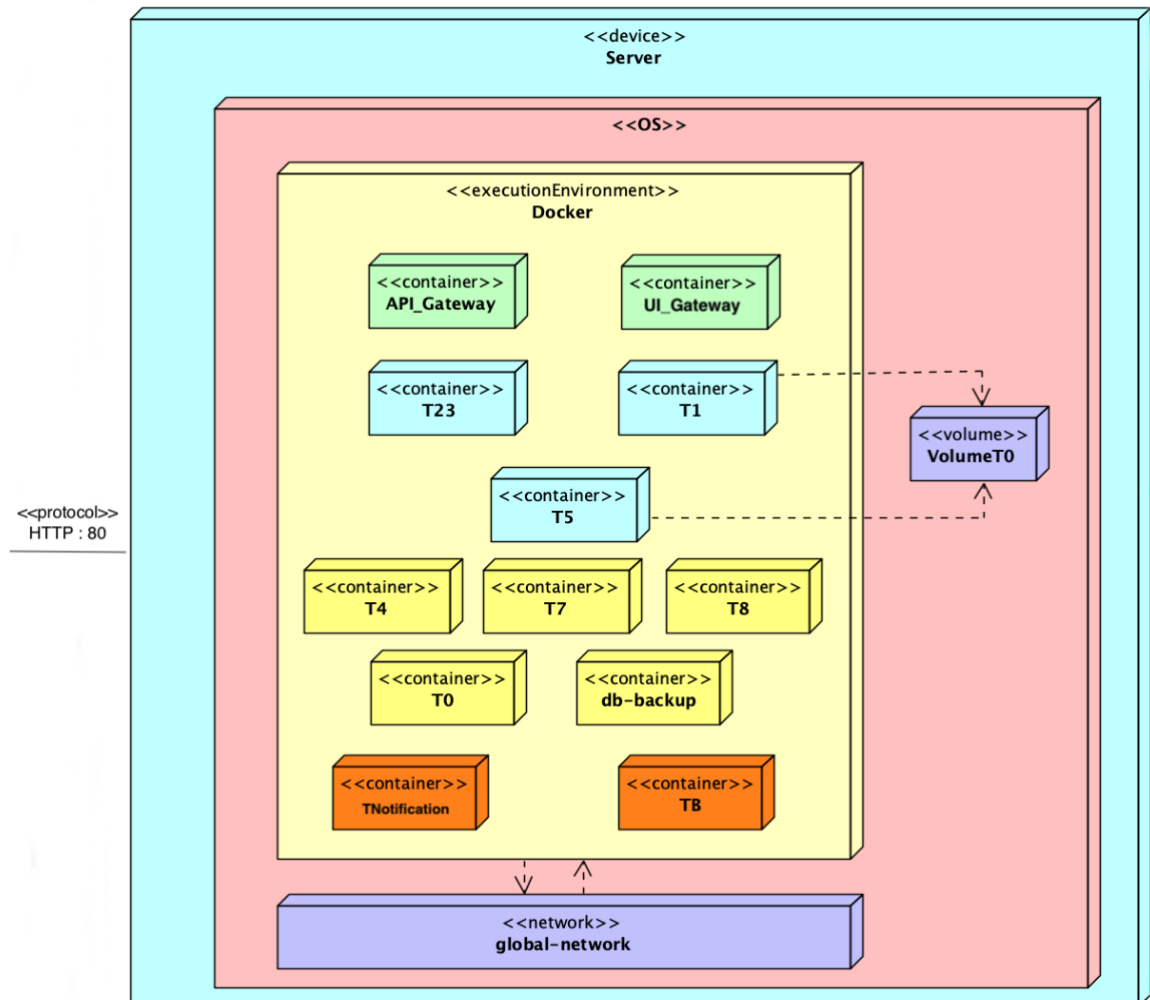
Modulo / Microservizio	Tipo intervento	Descrizione sintetica del modulo	Motivazione principale
TNotification (Notification Service)	Nuovo	Microservizio dedicato alla gestione completa delle notifiche (ricezione eventi asincroni, persistenza, REST, SSE)	Separare il dominio notifiche da T23 e dagli altri servizi, migliorando scalabilità e manutenibilità.
T-Shared-Notification (libreria)	Nuovo	Libreria condivisa che definisce DTO, costanti e modalità di invio delle richieste di creazione notifica verso TNotification tramite RabbitMQ	Eliminare duplicazioni di DTO/logica e fornire un contratto standard ai produttori di notifiche
TB (Broker)	Nuovo	Aggiunta del Message Broker in un container dedicato che sarà raggiunto dagli altri servizi per lo scambio dei messaggi in maniera asincrona.	Permettere la comunicazione asincrona tra microservizi, evitando di bloccare/rallentare l'elaborazione della loro business logic
API_Gateway	Modificato	Punto di ingresso unificato per tutte le chiamate HTTP del sistema	Aggiunta nuova rotta per il servizio TNotification

Il seguente diagramma Component & Connector fornisce una vista strutturale dell'architettura, nella quale sono evidenziati i nuovi componenti introdotti, il microservizio TNotification, il Broker e le relative interfacce esposte. Il diagramma consente di comprendere il ruolo dei componenti all'interno del sistema e le modalità con cui essi si integrano con il resto dell'architettura.



4.3 Deployment Diagram

Il diagramma di deployment riportato di seguito descrive la distribuzione fisica dei servizi all'interno dell'infrastruttura. In particolare, sono evidenziati i due nuovi container messi a disposizione all'interno del sistema.



4.4 Descrizione dei test

Il microservizio di notifiche è stato testato utilizzando strumenti e approcci differenti per il backend e per il frontend, al fine di validare in modo indipendente i due principali canali di interazione del sistema: la comunicazione asincrona tra microservizi e l'esposizione delle API verso il client.

4.4.1 Testing lato Backend (integrazione asincrona)

Per il testing lato backend è stato simulato un microservizio produttore di messaggi, configurato per utilizzare la libreria condivisa T-Shared-Notification e inviare eventi di notifica verso il message broker RabbitMQ.

Questo approccio ha permesso di verificare:

- il corretto utilizzo della libreria client da parte dei servizi produttori;
- la corretta serializzazione e deserializzazione dei messaggi (NotificationDTO);
- il funzionamento della topologia di messaggistica (exchange, routing key, coda);
- la persistenza delle notifiche nel database del microservizio TNotification;
- l'invio degli eventi verso i client sottoscritti tramite SSE.

4.4.2 Testing lato Frontend (API REST e SSE)

Per il testing lato frontend sono stati utilizzati **strumenti standard come cURL**, che hanno permesso di interagire direttamente con le API REST e con il canale Server-Sent Events esposto dal microservizio di notifiche tramite API Gateway.

In particolare, cURL è stato utilizzato per:

- recuperare lo storico delle notifiche;
- verificare il filtraggio nel recupero delle notifiche;
- segnare notifiche come lette;
- eliminare singole notifiche o l'intero storico;
- sottoscrivere al canale SSE e osservare la ricezione delle notifiche in tempo reale.

Esempi di utilizzo

1. Visualizza notifiche

Endpoint: GET /notifications/{userId}

```
lukemu@MacBookAir T9 % curl -v http://localhost:8089/notifications/42 | json_pp
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--    0*   Trying [::1]:8089...
* Connected to localhost (::1) port 8089
> GET /notifications/42 HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Mon, 29 Dec 2025 15:52:25 GMT
<
{ [7149 bytes data]
100 7141    0 7141    0    0 19128    0 --:--:-- --:--:-- --:--:-- 19144
* Connection #0 to host localhost left intact
[
  {
    "body" : "Complimenti, hai sbloccato l'obiettivo 'Primo test completato'.",
    "createdAt" : "2025-12-29T15:43:42.822901Z",
    "id" : 39,
    "read" : false,
    "title" : "Achievement sbloccato!",
    "type" : "ACHIEVEMENT_UNLOCKED",
    "userId" : 42
  },
  {
    "body" : "Complimenti, hai sbloccato l'obiettivo 'Primo test completato'.",
    "createdAt" : "2025-12-29T14:50:50.769114Z",
    "id" : 38,
    "read" : false,
    "title" : "Achievement sbloccato!",
    "type" : "ACHIEVEMENT_UNLOCKED",
    "userId" : 42
  },
]
```

2. Visualizza notifica

Endpoint: GET /notifications/42/12

```
lukemu@MacBookAir T9 % curl -v http://localhost:8089/notifications/42/12 | json_pp
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0  0 --:--:-- --:--:-- --:--:--    0*   Trying [::1]:8089...
* Connected to localhost (::1) port 8089
> GET /notifications/42/12 HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Content-Type: application/json
< Transfer-Encoding: chunked
< Date: Mon, 29 Dec 2025 16:23:09 GMT
<
{ [213 bytes data]
100  207    0  207    0    0 3880    0 --:--:-- --:--:-- --:--:-- 3905
* Connection #0 to host localhost left intact
{
  "body" : "Complimenti, hai sbloccato l'obiettivo di test numero 2.",
  "createdAt" : "2025-12-09T23:27:20.761701Z",
  "id" : 12,
  "read" : false,
  "title" : "Achievement sbloccato #2",
  "type" : "ACHIEVEMENT_UNLOCKED",
  "userId" : 42
}
```

3. Elimina notifica

Endpoint: DELETE /notifications/42/11

```
lukemu@MacBookAir T9 % curl -v -X DELETE http://localhost:8089/notifications/42/11
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8089...
* Connected to localhost (::1) port 8089
> DELETE /notifications/42/11 HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 204
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Date: Mon, 29 Dec 2025 16:25:47 GMT
<
* Connection #0 to host localhost left intact
```

4. Elimina tutte le notifiche

Endpoint: DELETE /notifications/42

```
[lukemu@MacBookAir T9 % curl -v -X DELETE http://localhost:8089/notifications/42
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8089...
* Connected to localhost (::1) port 8089
> DELETE /notifications/42 HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 204
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Date: Mon, 29 Dec 2025 16:26:29 GMT
<
* Connection #0 to host localhost left intact
```

5. Segna notifica come letta

Endpoint: PATCH /notifications/42/11/read

```
lukemu@MacBookAir T9 % curl -v -X PATCH http://localhost:8089/notifications/42/11/read
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying [::1]:8089...
* Connected to localhost (::1) port 8089
> PATCH /notifications/42/11/read HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 204
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Date: Mon, 29 Dec 2025 16:24:55 GMT
<
* Connection #0 to host localhost left intact
```

6. Sottoscrizione SSE

Endpoint: GET /notifications/subscribe/{userId}

```
lukemu@MacBookAir ~ % curl -N -v -H "Accept: text/event-stream" http://localhost:8089/notifications/subscribe/42
* Host localhost:8089 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
* Trying ::1:8089...
* Connected to localhost (::1) port 8089
> GET /notifications/subscribe/42 HTTP/1.1
> Host: localhost:8089
> User-Agent: curl/8.7.1
> Accept: text/event-stream
>
* Request completely sent off
< HTTP/1.1 200
< Vary: Origin
< Vary: Access-Control-Request-Method
< Vary: Access-Control-Request-Headers
< Content-Type: text/event-stream
< Transfer-Encoding: chunked
< Date: Mon, 29 Dec 2025 15:43:43 GMT
<
event:NOTIFICATION
data:{"id":39,"userId":42,"type":"ACHIEVEMENT_UNLOCKED","title":"Achievement sbloccato!","body":"Complimenti, hai sbloccato l'obiettivo 'Primo test completato'.","read":false,"createdAt":"2025-12-29T15:43:42.822900930Z"}
```

4.5 Anti-pattern rilevati

Di seguito sono riportati i principali anti-rilevati prima e durante lo sviluppo, con le soluzioni attuate nel correggerli:

- **Macro servizio con responsabilità multiple (T23):**

Nella versione precedente dell'architettura, la gestione delle notifiche era parzialmente incorporata nel servizio T23, che si occupa già di autenticazione e gestione utenti.

Questa scelta tendeva a rendere T23 un “macro servizio” con responsabilità eterogenee (autenticazione, profilo, notifiche), andando contro il principio di singola responsabilità e delle architetture a microservizi, rendendo quindi più complessa l'evoluzione indipendente delle funzionalità, aumentando il rischio di regressioni e rendendo più difficile scalare in modo selettivo solo la parte maggiormente sollecitata.

Per correggere questo problema è stata presa la decisione di deputare la gestione delle notifiche ad un microservizio dedicato (TNotification), lasciando a T23 il solo ruolo di servizio di autenticazione/identity e separando nettamente il dominio “notifiche” dal dominio “utenti”.

Contestualmente, è stata introdotta una libreria condivisa (T-shared-notification), che fornisce un punto di accesso standard per l'invio di eventi di notifica verso TNotification; in questo modo la creazione delle notifiche non è più responsabilità diretta dei singoli servizi, ma viene delegata al microservizio dedicato tramite un contratto comune.

- **Duplicazione di DTO:**

In una prima fase, i dati delle notifiche (DTO) erano duplicati in più microservizi, creando versioni multiple e potenzialmente incoerenti della stessa struttura informativa. Questo tipo di duplicazione è un altro “bad smell” frequente nelle architetture distribuite, perché aumenta il rischio di disallineamento tra servizi quando un campo viene aggiunto o modificato.

Si è quindi scelto di correggere questo smell tramite la centralizzazione del contratto nella libreria condivisa. Essa è stata progettata proprio per eliminare questa duplicazione: definisce un unico contratto di messaggistica (DTO, costanti, producer) che tutti i servizi possono utilizzare, mantenendo comunque TNotification come unico responsabile della persistenza e della logica di business sulle notifiche. In questo modo si riduce l'accoppiamento, si facilita l'evoluzione del formato dei messaggi e si evita di ricadere in un “monolite distribuito” basato su copie divergenti delle stesse strutture dati.

- **Comunicazione Sincrona e Bloccante tra Servizi:**

Inizialmente, l'interazione tra i servizi di gioco e il sistema di notifiche era concepita in modo sincrono: il microservizio produttore inviava una richiesta e attendeva una risposta immediata di avvenuta ricezione.

Questa modalità introduce un forte accoppiamento temporale tra i sistemi: se il servizio di notifiche subisce rallentamenti o diviene indisponibile, il blocco si propaga a cascata sul servizio chiamante (es. gestione partita), degradando l'esperienza utente o addirittura interrompendo il flusso di gioco. Inoltre, gestire logicamente le notifiche come chiamate bloccanti contraddice la natura stessa di un evento "informativo", che non dovrebbe mai ostacolare le operazioni critiche del core business.

La correzione di questa problematica è avvenuta tramite il passaggio a un'architettura **Event-Driven asincrona**. È stato introdotto un **Message Broker** (RabbitMQ) e la modalità di comunicazione predefinita è stata convertita nel pattern Fire-and-Forget: i microservizi produttori ora pubblicano l'evento di notifica e proseguono immediatamente la loro elaborazione, senza attendere conferme sincrone.

La libreria condivisa T-shared-notification supporta comunque la modalità *Request-Reply* per casi eccezionali, ma l'architettura favorisce ora il totale disaccoppiamento, garantendo che picchi di carico o malfunzionamenti del servizio notifiche non impattino mai sulla stabilità dei servizi di gioco.

5. Sviluppi futuri

Nel prosieguo del progetto saranno possibili alcune evoluzioni funzionali e strutturali del microservizio TNotification:

- **Gestione degli utenti tramite autenticazione JWT**

Attualmente l'identificativo dell'utente (``userId``) viene ricavato direttamente dai parametri della richiesta HTTP, ma in un'architettura completamente integrata con il sistema di autenticazione sarà necessario estrarre questi dati direttamente dal token JWT associato alla sessione. In questo modo si migliorerà la sicurezza e si eviterà di affidarsi a valori passati manualmente dal client, riducendo il rischio di inconsistenze o manipolazioni.

- **Gestione delle eccezioni REST**

Un altro aspetto importante è il refactoring della gestione delle eccezioni. Al momento il service lancia errori generici (ad esempio ``IllegalArgumentException``), che producono risposte HTTP 500. In futuro sarà opportuno introdurre un sistema centralizzato di error handling, capace di restituire codici di stato e messaggi più chiari (ad esempio 404 per "notifica non trovata", 403 per accessi non autorizzati, 400 per input errati, ecc.). Questo permetterà di rendere il servizio più robusto e conforme alle best practice REST.

- **Aggiunta di nuovi endpoint REST**

Un'altra estensione naturale riguarda l'esposizione di nuovi endpoint REST, nel caso in cui il frontend richieda funzionalità aggiuntive (ad esempio endpoint per il conteggio delle notifiche non lette, paginazione, filtri avanzati).

- **Resilienza e Affidabilità della Messaggistica Asincrona**

Attualmente, l'integrazione con il broker (RabbitMQ) gestisce il flusso standard di invio e ricezione. In particolare, il broker rappresenta un punto critico per l'affidabilità complessiva del sistema, in quanto un suo malfunzionamento o indisponibilità potrebbe compromettere la corretta propagazione degli eventi e, di conseguenza, la creazione delle notifiche.

Un'evoluzione fondamentale riguarderà l'irrobustimento di questo meccanismo per garantire la data consistency anche in caso di guasti infrastrutturali. Questo può essere fatto tramite Retry Policy e Dead Letter Queue (DLQ), cioè implementando politiche di retry automatico per i messaggi falliti e configurando code "morte" (DLQ) dove parcheggiare i messaggi non processabili per un'analisi successiva, evitando la perdita di dati.

- **Estensione della gerarchia di notifiche**

Inoltre, potrà essere introdotta una gerarchia di tipi di notifica più ricca per particularizzare attributi e metodi in base alla categoria della notifica. Ad esempio, in base al `type` della notifica, il client potrebbe essere reindirizzato automaticamente a schermate diverse (ad esempio notifiche di messaggio, notifiche di sistema, notifiche legate a una specifica partita o esercizio).

Tale modifica sarebbe peraltro agevolata dalla scelta di PostgreSQL, che offre un campo JSONB nel caso in cui ci fossero attributi specifici legati al tipo della notifica da salvare.